

中山大学计算机学院本科生实验报告

(2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

年级	23 级	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇
电话	13421145433	Email	linhy69@mail2.sysu.deu.cn
开始日期	2024.12.13	完成日期	2024.12.13

第一题

1. 实验题目

DAG

2. 实验目的

输入一个有向图, 判断该图是否是有向无环图(Directed Acyclic Graph)。

3. 算法设计

1. 判断是否为有向无环图, 即判断是否可以拓扑排序输出所有结点
2. 使用矩阵来存放有向图, 同时使用一个数组来存放拓扑排序输出的结点
3. 拓扑排序: ①找到矩阵有向图矩阵中全为 0 的一列 i , 即找到入度为 0 的结点
②将 i 行的值全部置 0, 即删除 i 的所有出度
③重复上述操作 4. 当结果数组的大小等于结点数则表示可以拓扑排序, 为有向无环图

4. 程序运行与测试

1. 输入数据, 创建矩阵无向图, 创建存放输出结点的结果数组

```

int n,m;
cin>>n>>m;
vector<vector<int>> graph(n+1,vector<int>(n+1,0));
vector<int> res;//用来存放输出的结点
while(m--)
{
    int a,b;
    cin>>a>>b;
    graph[a][b]=1;
}

```

2. 拓扑排序过程一：找到入度为 0 即列全为 0 的结点（同时可作判断，不满足的条件直接终止程序结束）

```

for(int i=1;i<=n;i++)
{
    int flag=1;
    for(int j=1;j<=n;j++)
    {
        if(graph[j][i]==1)
        {
            flag=0;
            break;
        }
    }
    if(flag && find(res.begin(),res.end(),i)==res.end())//找到下一个
    {
        begin=i;
        break;
    }
}
if(find(res.begin(),res.end(),begin)!=res.end())//找不到下一个了
{
    cout<<"0"<<endl;
    return 0;
}

```

3. 拓扑排序过程二：将输出的结点的出度全部删掉

```

//把begin出去的边删掉
for(int i=1;i<=n;i++)
    graph[begin][i]=0;
res.push_back(begin);//把begin放进去

```

4. 程序终止条件：结果数组放满了结点

```

if(res.size()==n+1)
{
    cout<<"1"<<endl;
    return 0;
}

```

5. 实验总结与心得

1. 实验题难度较小，就是拓扑排序的两种方法（使用矩阵图表示和邻接图表示）的第一种的解决方法
2. 首先是理解题目，将判断有向无环图转化为拓扑排序问题
3. 然后再思考如何进行拓扑排序，结合矩阵图表示的特点，找入度为 0 的结点即列全为 0，删除出度边即将行全部置 0

第二题

1. 实验题目

Euler Euler

2. 实验目的

Description:

Given an undirected graph G , decide whether G has an Euler Circuit, Euler Path, or neither.

Input:

There are multiple test cases. The first line contains an integer T , indicating the number of test cases. Each test case begins with a line containing two integers, where n is the number of nodes (numbered from 1 to n) and m is the number of edges. In the following m lines, each line has two integers indicating two adjacent nodes of an edge.

Output:

For each test case, print a string Euler Circuit, Euler Path, or Neither.

3. 算法设计

1. 理论：①若为连通图，且有两个度为奇数的结点，其他全为度为偶数的结点，是欧拉路径 ②若为连

通图，且全为度为偶数的结点，是欧拉通路 ③其他则为 Neither

2. 判断要素：①是否为连通图 ②度数为奇数的结点个数 ③度数为偶数的结点个数

3. 判断连通图：使用邻接表表示无向图，进行 DFS 搜索，如果有结点未被访问则不连通

4. 记录度数：这里的拓扑排序用邻接表表示图，用数组来记录每一个结点的度数

5. 拓扑排序：①创建邻接表图、创建度数的数组、输入 ②判断连通图是否连通，不连通直接输出 Neither 结束 ③判断度为奇数的结点是否为 2，欧拉路径；判断度为奇数的结点是否为 0，欧拉回路；其他情况 Neither

4. 程序运行与测试

1. 创建邻接表图、创建度数的数组、输入

```
int n, m;
cin >> n >> m;

vector<vector<int>> adj(n + 1);
vector<int> degree(n + 1, 0);

//读取边的信息
while (m--) {
    int a, b;
    cin >> a >> b;
    if (a == b)
        continue;
    adj[a].push_back(b);
    adj[b].push_back(a);
    degree[a]++;
    degree[b]++;
}
```

2. 判断是否为连通图

```

void dfs(int v, vector<vector<int>>& adj, vector<bool>& visited)
{
    visited[v] = true;
    for (int u : adj[v])
        if (!visited[u])
            dfs(u, adj, visited);
}

bool isConnected(int n, vector<vector<int>>& adj)
{
    vector<bool> visited(n + 1, false);
    dfs(1, adj, visited);
    //从顶点1开始DFS

    //如果有未访问的顶点 说明图不连通
    for (int i = 1; i <= n; i++)
        if (!visited[i])
            return false;
    return true;
}

```

3.判断度数的关系输出结果

```

int count_odd = 0;
for (int i = 1; i <= n; i++)
    if (degree[i] % 2 != 0)
        count_odd++;

if (count_odd == 2)
    cout << "Euler Path" << endl;
else if (count_odd == 0)
    cout << "Euler Circuit" << endl;
else
    cout << "Neither" << endl;

```

5. 实验总结与心得

1. 个人纠结的点在于，做题时忘记判断是否连通了，要利用 DFS 搜索来判断是否连通，是否连通与度数直接没有关系
- 2.此题时拓扑排序的另一种解法（和题目一向对应），使用邻接表和度数的数组进行求解，因此再输入时还要多一个对度数的数组的处理

第三题

1. 实验题目

Ordering Tasks

2. 实验目的

John has n tasks to do. Unfortunately, the tasks are not independent and the execution of one task is only possible if other tasks have already been executed.

3. 算法设计

1. 拓扑排序的解法同题目二类似，在处理上不同，同样是创建邻接表和数组，还要创建度数为 0 的数组
2. 拓扑排序的排序处理：①通过度数的数组，找到度数为 0 的结点 ②将结点输出 ③将所有邻接的结点的度数-1 ④重复上述操作
3. 在处理过程中的易错点：题目要求按照字典序最小的方式输出，则每次有多个入度为 0 的结点时，输出最小的，因此可以使用优先队列来存储度数为 0 的结点

4. 程序运行与测试

1. 创建邻接表表示图、创建表示入度的数组、创建表示入度为 0 结点的数组、输入

```
int n, m; //n任务数量 m任务间的依赖关系数量
cin >> n >> m;

vector<vector<int>> graph(n + 1);
vector<int> InD(n + 1, 0); //入度数组

while (m--) {
    int a, b;
    cin >> a >> b; //a必须在b之前执行
    graph[a].push_back(b);
    InD[b]++;
}
//使用 priority_queue<int, vector<int>, greater<int>> pq, 这个优先队列会始终弹出最小的元素
(即字典序最小的任务)。
priority_queue<int, vector<int>, greater<int>> InDE0; //入度为0
for (int i = 1; i <= n; i++) {
    if (InD[i] == 0) {
        InDE0.push(i);
    }
}
```

2. 进行拓扑排序的过程

```

vector<int> res; //任务执行顺序
while (!InDE0.empty()) {
    int begin = InDE0.top();
    InDE0.pop();
    res.push_back(begin);

    // 执行当前任务并更新其他任务的入度
    for (int i = 0; i < graph[begin].size(); i++)
    {
        InD[graph[begin][i]]--;
        if (InD[graph[begin][i]] == 0)
            InDE0.push(graph[begin][i]);
    }
}

```

5. 实验总结与心得

1. 实验题难度不大，个人纠结的点在于，疏忽了题目中说的按照字典序最小的方式输出
2. 要熟悉优先队列的使用，在数据结构与算法的解题中经常用到
3. 熟悉拓扑排序的操作：①通过度数的数组，找到度数为 0 的结点 ②将结点输出 ③将所有邻接的结点的度数-1 ④重复上述操作